

Who Wrote It?

Identifying LLMs from Their Responses

CMPSC 448 Course Project

Team leader and sole member

Shaun Gulati

Individual project. I am not presenting.
The default presentation score applies.

October 11, 2026

CMPSC 448: Machine Learning and Algorithmic AI
Instructor: Wenpeng Yin
The Pennsylvania State University

Contents

1 Problem	2
2 Dataset	2
2.1 Where the text came from	2
2.2 A hole I had to close	3
2.3 Task labels	3
2.4 Balance and the split	3
3 Models and training	4
3.1 Words and vectors	4
3.2 CNN	5
3.3 LSTM	5
3.4 How I picked the epoch	5
3.5 Baselines from Chapter 1	6
3.6 How I score a prediction	6
4 RQ1: the response alone	6
5 RQ2: does the instruction help?	8
5.1 Why the instruction could have worked	9
6 RQ3: a task the model has not seen	10
7 RQ4: what the models are doing differently	11
7.1 Counts I can see without a classifier	11
7.2 Logistic regression weights	13
7.3 K-means, from Chapter 2	13
7.4 Deleting signals and retraining	13
8 Lessons	14
9 LLM tools	15
10 How to rerun	15

1 Problem

The question in this project is simple. Given text that a language model wrote, can I tell which model wrote it?

I treat this as supervised learning, the way Chapter 1 sets it up. Each example is a pair (x, y) . The gold label y is the model name. The input x is some text: the response, the user’s instruction, or both. I want a function $f(x)$ that predicts y .

There are four labels, so a guess would be right about one time in four. Accuracy of 0.25 is the chance line. The four models are balanced, so that number is fair.

I looked at four questions:

- **RQ1.** If x is only the response, how well can a CNN and an LSTM name the model?
- **RQ2.** Does the user’s instruction help? I compare the instruction alone, the response alone, and the two concatenated. Chapter 7 trains instruction-tuned models on a concatenated (instruction, input) pair. I used the same idea, with the response in the input slot.
- **RQ3.** If I train on some tasks and test on a task I left out, does the score drop? Chapter 6 talks about a source task S and a target task T , and says labeled data from S may not adapt well to a new domain. Chapter 7 says the instructions at train time and test time should match.
- **RQ4.** What makes the responses look different? Chapter 7 (instruction tuning) and Chapter 8 (RLHF) are the course explanation for why models pick up different habits. I measure those habits and then delete some of them and retrain.

Every number below comes from one run with seed 42. The file is `results/metrics.json`.

2 Dataset

2.1 Where the text came from

I used UltraFeedback [1], the public set at [openbmb/UltraFeedback](https://openbmb.github.io/UltraFeedback/). The license is MIT. I did not call any model API, and I did not use my own chats.

UltraFeedback took prompts from ShareGPT, UltraChat, Evol-Instruct, TruthfulQA, FalseQA, and FLAN. For each prompt it asked four models from a larger pool to answer. I kept one model from each of four families:

Label	Family	Who trained it
<code>bard</code>	Bard	Google
<code>falcon-40b-instruct</code>	Falcon	Technology Innovation Institute
<code>gpt-4</code>	GPT	OpenAI
<code>llama-2-70b-chat</code>	Llama	Meta

I left out WizardLM, Vicuna, and UltraLM because those are Llama models with extra training. I wanted the family itself to be different. I left out StarChat because it is a code model, so a code prompt would almost give the label away.

I dropped responses shorter than 40 characters, responses that were not text, and responses that were mostly not Latin letters. I dropped instructions shorter than 12 characters. The released files do not include the `id` field from the dataset card, so I hashed the instruction and used that as the prompt id.

2.2 A hole I had to close

Bard has no answers on three FLAN subsets: `flan_v2_niv2`, `flan_v2_p3`, and `flan_v2_flan2021`. That is 17,939 prompts. Those prompts use a very fixed template. If I had kept them, two things would go wrong. The instruction alone could rule out Bard. The responses would also look different, because those templates ask for short answers and Bard never wrote them. That is not a writing style. It is a missing row in the table. I dropped those sources. On the sources I kept, all four models were actually queried.

2.3 Task labels

RQ3 needs a task. UltraFeedback does not give one, so I labeled each instruction with rules in `src/domains.py`. The label does not look at the response. I check code first, then writing, then reasoning, then question answering. I dropped prompts that matched none of these (9,681 prompts).

- **Code.** A code fence, a language name, or a phrase like “write a function.” I did not use the word “function” by itself. It shows up in normal sentences.
- **Writing.** A request for a story, poem, essay, email, article, letter, speech, or summary.
- **Reasoning.** What is left of `flan_v2_cot`: entailment items, multiple choice, and step-by-step word problems. A clean “math only” bucket was too small once I stopped matching the word “proof” in ordinary sentences.
- **QA.** TruthfulQA, FalseQA, and remaining questions.

These rules are rough. A summary is not the same kind of writing as a short story. I froze the rules before I looked at test scores.

2.4 Balance and the split

Question answering was much larger than writing. Inside each task I kept the same number of rows for each model, and I capped that number at 700. The cell sizes are code 676, QA 700, reasoning 664, and writing 627 per model. That is 10,668 rows and 8,948 prompts.

Chapter 1 splits with `train_test_split` and a 20% test set. I also needed a validation set, because Chapter 9 says to look at the validation curve when you choose training settings. I used 70% train, 15% validation, and 15% test. The split is by prompt, and it is stratified by task. Every answer to one prompt stays in one split. No prompt id is in two splits. The counts are a little uneven across models because one prompt was not always answered by all four models.

Table 1: Rows by split and model.

Split	Bard	Falcon-40B	GPT-4	Llama-2-70B
Train	1884	1884	1877	1837
Val	398	374	413	411
Test	385	409	377	419

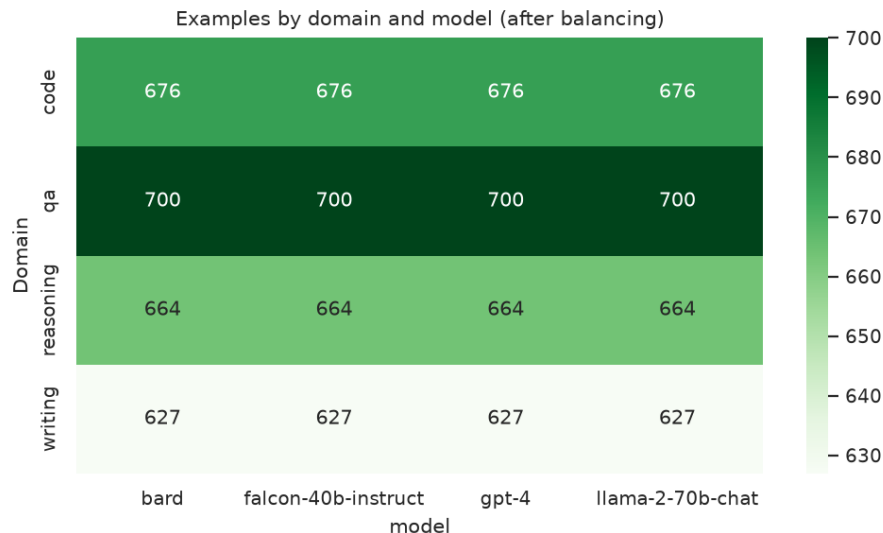


Figure 1: Counts by task and model after balancing. Within a task the four models match.

I stored at most 6,000 characters of each response. Almost no rows hit that cap. When I talk about length in RQ4, I use the word count from before the cap.

UltraFeedback also wrapped each call in a random “principle” prompt, such as helpfulness or honesty. I never show that prompt to the classifier. The response was still generated under it. I come back to that in RQ4.

3 Models and training

Both neural models are many-to-one classifiers, like the sentiment example in Chapter 4. A sequence of word vectors goes in. One label comes out. The scores go through a softmax, and the loss is negative log-likelihood, as in Chapter 9. In PyTorch that is `CrossEntropyLoss`. The optimizer is Adam, which Chapter 9 calls the usual choice. The learning rate is $\alpha = 10^{-3}$. The batch size is $|B| = 64$. Chapter 9 says a batch size is often a power of two and somewhere between 1 and a few hundred. I did not do a batch-size search. The validation curve for the CNN rose and then flattened, so I left $|B|$ alone.

I did not use dropout or weight decay. We did not cover those. I also did not clip gradients. On one ablation the CNN loss blew up anyway. I say more about that in the lessons.

3.1 Words and vectors

I split text into words and punctuation and then lowercase it. Chapter 6 says each word gets one vector. I learned that table while training. The vocabulary is the 12,000 most common training words that appear at least twice, plus a pad token, an unknown token, and a separator. Validation and test words that were not in training become unknown. I did not load GloVe or word2vec. Those are the static embeddings from Chapter 6, one vector per word for every context. Mine is the same kind of table, but it is learned on this dataset.

Sequences are cut at 128 tokens. For the concatenated view I keep at most 40 tokens of the instruction and 87 tokens of the response, with a separator in between. If I had cut the raw concatenation at 128, a long instruction would have deleted the response.

3.2 CNN

Chapter 4 builds a CNN as convolution, then ReLU, then max-pooling, then a fully connected layer. The 1D example uses a width-3 filter and zero padding on both ends. I did that. The input to the convolution is the sequence of 128-dimensional word vectors. I use 96 filters, so there are 96 activation maps. Each map is a channel, which the lecture calls one version of the representation. After ReLU I take the max over time. That is the “most representative” feature from the pooling slides. A linear layer turns the 96 numbers into 4 scores.

The lecture’s CNN-versus-RNN slide says a CNN is fast, sees left and right context, and only sees local context. That is the model I report as the CNN.

I also tried filter widths 3, 4, and 5 together, on the output-only task only. Chapter 4 never discusses more than one width. I found that setup outside of class. It is an extra comparison, not the main CNN.

3.3 LSTM

The required RNN is the LSTM from Chapter 4, in one direction. The lecture writes one step as

$$\begin{aligned}i_t &= \sigma(x_t U^i + h_{t-1} W^i + b_i) \\f_t &= \sigma(x_t U^f + h_{t-1} W^f + b_f) \\o_t &= \sigma(x_t U^o + h_{t-1} W^o + b_o) \\q_t &= \tanh(x_t U^q + h_{t-1} W^q + b_q) \\p_t &= f_t * p_{t-1} + i_t * q_t \\h_t &= o_t * \tanh(p_t).\end{aligned}$$

i_t , f_t , and o_t are the input, forget, and output gates. q_t is the candidate update. p_t is the cell state. h_t is the hidden vector the rest of the network uses. U multiplies the current input and W multiplies the previous hidden state. The same weights are used at every time step.

PyTorch’s `nn.LSTM` is this recurrence. I use one layer and a hidden size of 128. For a many-to-one model the label is read from h_T , the hidden state at the end of the real text. Padding is not a word. If I had let the pad steps run, they would have overwritten h_T . I pack the batch so the recurrence stops at the last real word. We did not cover packing in class. It is only there so different lengths can share a batch.

Chapter 4 says this kind of RNN only sees one side, but it can carry information a long way. It is also slow because the steps are in order. I felt that while training. The CNN epochs were quick. The LSTM epochs were not.

I also trained a bidirectional LSTM on the output-only task. That is two of the same LSTMs, one in each direction. We did not cover it. I tried it because the lecture lists “one side” as the limitation. It is an extra, not the required model.

3.4 How I picked the epoch

I trained for at most 12 epochs. My first LSTM runs were still improving at epoch 6, so a cap of 6 was too low. After each epoch I scored the validation set. I kept the weights from the epoch with the best validation accuracy, which is the number Chapter 1 uses. If accuracy did not improve for 3 epochs, I stopped. The test set was not used for that choice. The CNN’s chosen epoch on the output-only run was epoch 8 (validation accuracy 0.748). The LSTM’s was epoch 7 (validation accuracy 0.623).

3.5 Baselines from Chapter 1

The neural nets are not the only models in the class. Chapter 1 covers Naive Bayes and logistic regression. I used both, on word counts. Each word is a feature X_i . The count matrix is built on the training text only, with the same 12,000-word cap.

For Naive Bayes I used multinomial counts and add-one smoothing. Chapter 1 fixes zero counts with MAP and m virtual points. I set $m = 1$.

Logistic regression in class is a sigmoid on a linear score, for two classes. Slide 87 of Chapter 1 asks what the multi-class version should look like. I used one weight vector per class and a softmax, which is the same softmax as in Chapter 9. I did not add an extra penalty term. The solver did not fully converge on the instruction-only features. I say so in RQ2.

Chapter 1 calls the weights the importance of the features. I use the logistic regression weights in RQ4.

3.6 How I score a prediction

Accuracy is the fraction of test rows I got right. For a single class, Chapter 2 defines precision and recall from the counts n_{ij} . If class j is the gold label, precision is the share of my predicted- j rows that really are j , and recall is the share of true j rows that I found. Then F mixes those two. Chapter 2 averages F over the clusters, $F = (1/r) \sum_i F_i$. I do the same average over the four models. This average is often called macro-F1. I will just call it average F . The count tables in the figures are the n_{ij} tables, with the true model on the rows and my prediction on the columns.

4 RQ1: the response alone

On the mixed test set (1,590 responses), the CNN is clearly above chance. The one-direction LSTM is above chance too, but it is much weaker. Logistic regression on word counts is close to the CNN. Naive Bayes is the weak baseline. Table 2 has the scores.

Table 2: RQ1 test scores. The input is the response only.

Model	Accuracy	Average F
CNN (width 3)	0.745	0.745
LSTM (one direction)	0.590	0.588
Logistic regression	0.731	0.732
Naive Bayes	0.563	0.560
Bidirectional LSTM (extra)	0.717	0.717
CNN with widths 3, 4, and 5 (extra)	0.767	0.767

I was surprised by the gap between the CNN and the LSTM. I expected the LSTM to be at least as good, because Chapter 4 says it can use long context. On this data the useful cues look local: a phrase near the start, a punctuation habit, a markdown mark. The CNN is built for that. The LSTM has to push the cue all the way to h_T . It also only reads left to right. The bidirectional extra supports that reading. Its accuracy is 0.717, well above the one-direction LSTM at 0.590. The three filter widths help a little (0.767 versus 0.745). They are not required to beat chance.

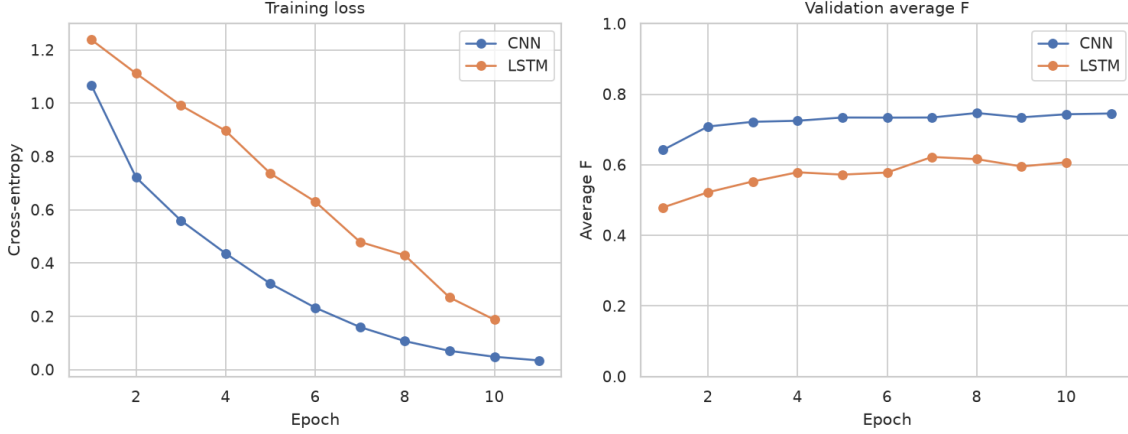


Figure 2: Output-only training. The CNN validation score flattens. The LSTM training loss keeps falling while validation average F stalls near 0.62.

The LSTM’s training loss at the chosen epoch is 0.479, and it keeps falling after that, while validation accuracy does not. That is overfitting. The CNN’s training loss at its chosen epoch is 0.107, and its test accuracy (0.745) matches its validation accuracy (0.748). The LSTM’s test accuracy (0.590) is lower than its validation accuracy (0.623). I do not have a neat explanation for that gap. The test set is only 1,590 rows, so some of it is noise. I still trust the ordering: CNN above LSTM.

Table 3: Per-class precision, recall, and F for the two required models. Supports are 385 (Bard), 409 (Falcon), 377 (GPT-4), and 419 (Llama-2-70B).

Model	Class	Precision	Recall	F
CNN	Bard	0.741	0.808	0.773
CNN	Falcon-40B	0.800	0.694	0.744
CNN	GPT-4	0.645	0.774	0.704
CNN	Llama-2-70B	0.820	0.709	0.761
LSTM	Bard	0.638	0.701	0.668
LSTM	Falcon-40B	0.611	0.599	0.605
LSTM	GPT-4	0.540	0.631	0.582
LSTM	Llama-2-70B	0.569	0.442	0.497

No CNN class falls apart. GPT-4 is the hardest for the CNN (F 0.704). Llama is the hardest for the LSTM (recall 0.442). Logistic regression is more even: its class F scores are 0.758, 0.712, 0.727, and 0.730. Naive Bayes is weakest on GPT-4 (F 0.490).

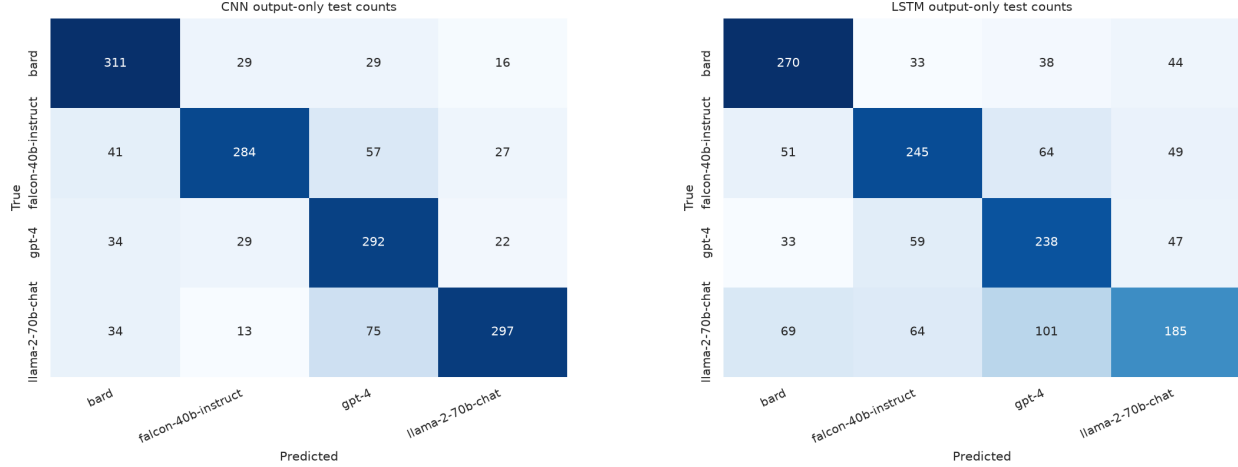


Figure 3: Output-only count tables. Rows are the true model.

In the CNN table, the biggest off-diagonal cell is 75 Llama responses predicted as GPT-4. In the LSTM table that cell is 101, and Llama’s diagonal is only 185 out of 419. The two long chat models are the ones that get confused. Falcon is more often sent to GPT-4 than to Bard (57 CNN errors on that edge).

The same CNN is not equally good on every task, even when every task was in training. Average F by task is code 0.743, QA 0.715, reasoning 0.844, and writing 0.661. The LSTM follows the same order at a lower level: 0.515, 0.552, 0.720, and 0.555. Reasoning is easy. On reasoning items the median lengths are 118, 55, 34, and 100 words for Bard, Falcon, GPT-4, and Llama. GPT-4 is much shorter there, so length is part of that score. Writing is the hard task. I use that in RQ3.

5 RQ2: does the instruction help?

No. Table 4 and Figure 4 compare the three views. “Both” is the Chapter 7 concatenation: instruction, then response.

Table 4: RQ2 test scores. Accuracy is on the left of each pair and average F is on the right.

View	CNN	LSTM	Log. reg.	Naive Bayes
Instruction only	0.265 / 0.231	0.254 / 0.253	0.255 / 0.255	0.250 / 0.231
Response only	0.745 / 0.745	0.590 / 0.588	0.731 / 0.732	0.563 / 0.560
Both	0.733 / 0.733	0.578 / 0.579	0.648 / 0.649	0.469 / 0.465

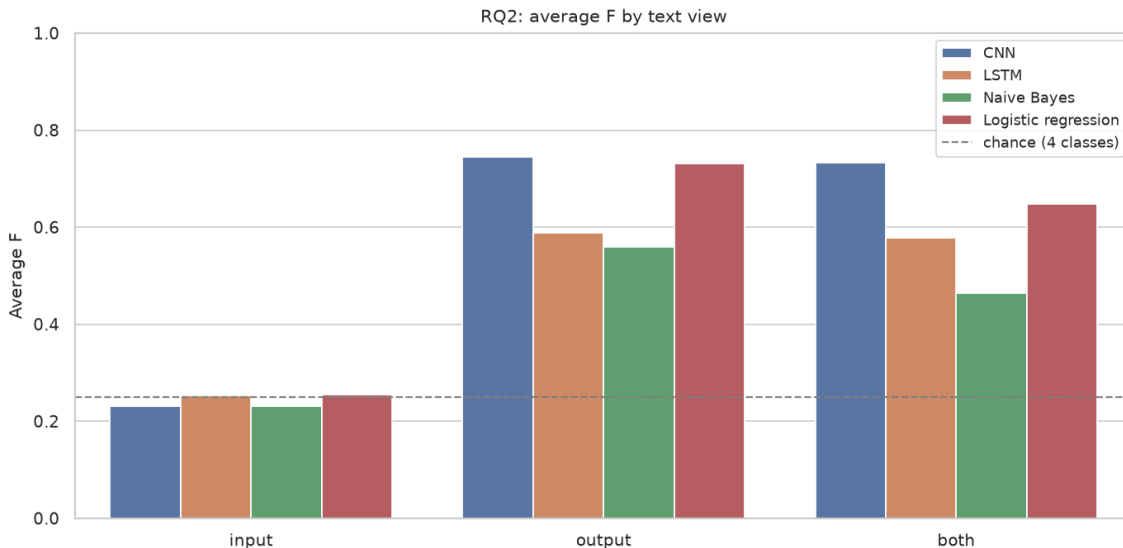


Figure 4: Average F for the instruction, the response, and both. The dashed line is chance for a balanced four-way F .

Instruction-only accuracy sits on 0.25. The CNN’s 0.265 is only a small step above it. A rough standard error for a 0.25 coin on 1,590 rows is about 0.011, so 0.265 is a bit high and the others are not. The logistic regression fit on the instruction did not converge in 500 steps. I tried 2,000 steps later. It still had not converged, and the accuracy was 0.264. I would not read that as a real signal.

Adding the instruction to the response makes every model worse. The CNN goes from 0.745 to 0.733. Logistic regression goes from 0.731 to 0.648. Naive Bayes goes from 0.563 to 0.469. The LSTM barely moves (0.590 to 0.578). The logistic regression drop is the clean one. Those instruction words are extra features, and the weights do not find a use for them.

5.1 Why the instruction could have worked

If users pick a model, or if a model was only online for part of the data collection, the instructions are not the same distribution for every label. Then the instruction predicts the model even before you see the answer. I could see a version of that in the raw files. The three FLAN subsets that Bard never answered have a stock template. Keeping them would have made the instruction useful. I removed them. After that, and after forcing each model to the same task counts, the instruction is almost useless.

There is still a small leftover. I downsampled each model separately, so the four models do not answer the exact same prompt set. That can create a tiny instruction-only lift. It is not “the user’s wording identifies the model.”

The useful signal is in the response. Chapter 7 and Chapter 8 say why that should be true. Instruction tuning and RLHF push each model toward a house style: how it starts, how long it goes, whether it hedges or itemizes. The instruction in this dataset was assigned across models. The style was not.

6 RQ3: a task the model has not seen

I used the Chapter 6 setup. Source tasks S are in training and validation. Target task T is held out. I did this twice. First T is writing (357 test rows) and S is code, QA, and reasoning (1,233 in-domain test rows). Then T is code (406 test rows) and S is everything else (1,184 in-domain test rows). The in-domain number and the cross-task number come from the same checkpoint.

Table 5: RQ3 average F . The drop is the in-domain score minus the score on the held-out task.

Held out	Model	In-domain	Held-out task	Drop
Writing	CNN	0.766	0.623	0.142
Writing	LSTM	0.577	0.510	0.067
Code	CNN	0.731	0.655	0.076
Code	LSTM	0.602	0.447	0.155

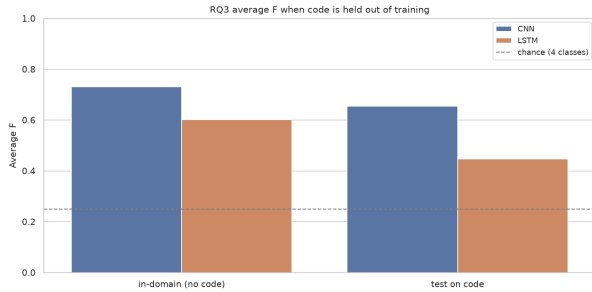
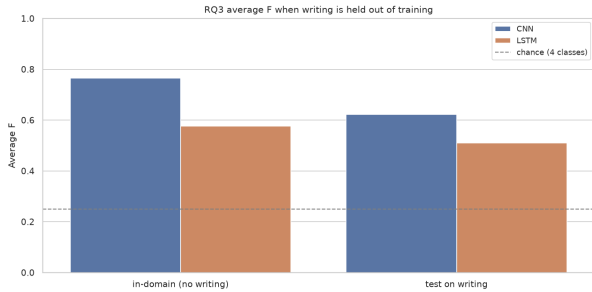


Figure 5: Average F when writing (left) or code (right) is left out of training.

The score drops, and it stays above 0.25. If the classifier had only memorized “what this task looks like” and had learned nothing about the model, the held-out task would be near chance. It is not. Some of the signal transfers. The drop means some of the in-domain score was about the task.

Recall by model shows that the transfer is uneven (Table 6).

Table 6: Recall by model, on the in-domain test rows and on the held-out task.

Setting		Bard	Falcon	GPT-4	Llama
Writing, CNN	in-domain	0.831	0.689	0.792	0.760
Writing, CNN	held out	0.722	0.682	0.581	0.512
Writing, LSTM	in-domain	0.658	0.561	0.613	0.489
Writing, LSTM	held out	0.767	0.477	0.538	0.302
Code, CNN	in-domain	0.789	0.680	0.829	0.640
Code, CNN	held out	0.635	0.641	0.745	0.591
Code, LSTM	in-domain	0.671	0.552	0.586	0.592
Code, LSTM	held out	0.302	0.573	0.480	0.438

When the held-out task is writing, Llama’s CNN recall falls from 0.760 to 0.512, and GPT-4’s falls from 0.792 to 0.581. Falcon does not really move. The LSTM’s Llama recall falls from 0.489 to 0.302. When the held-out task is code, the CNN drop is smaller, and the model that loses the most CNN recall is Bard (0.789 to 0.635). The LSTM’s Bard recall falls hard, from 0.671 to 0.302.

I would not over-read the smaller gaps. A few hundred test rows make a recall number noisy. The Llama writing drop and the LSTM’s Bard code drop are large enough that I believe them.

This matches the lecture’s warning. Chapter 6 says supervision from S may not fit T . Chapter 7 says a mismatch between train and test instructions hurts. Here the mismatch is the task, not the wording of a system prompt, but the effect is the same kind of thing. Falcon is the stable one in the CNN, and Falcon is short in every task (Section 7). A length cue does not care which task you held out. Llama’s essay style does.

7 RQ4: what the models are doing differently

Chapter 8’s story is that RLHF, after instruction tuning, pushes a model toward answers humans prefer. Those answers pick up habits: longer or shorter, more hedging, more lists, more “happy to help.” Chapter 7 says the instruction itself changes how a model behaves, and that models do not all follow the same instruction equally. UltraFeedback adds one more instruction on top, the random principle prompt. So some of what I measure is the base model, and some of it is how that model reacted to the same extra instruction.

7.1 Counts I can see without a classifier

Figure 6 and Table 7 describe the responses. Word counts use a letter-word pattern. For type-token ratio I use the first 50 words, and only if the response has 50 words. About 17% of rows are shorter. A raw type-token ratio is higher for short texts, so it would have made Falcon look more varied for a boring reason.

Table 7: Mean surface counts by model. Medians for length are in the text.

Statistic	Bard	Falcon	GPT-4	Llama
Words (mean)	260.1	117.8	230.1	268.8
TTR, first 50 words	0.715	0.761	0.770	0.757
Punctuation per word	0.190	0.170	0.231	0.200
Code fences	0.324	0.115	0.504	0.279
Bold markers	2.31	0.07	0.30	0.09
Bullet lines	2.85	1.07	3.08	3.00
Exclamation marks	0.24	0.25	0.38	0.86

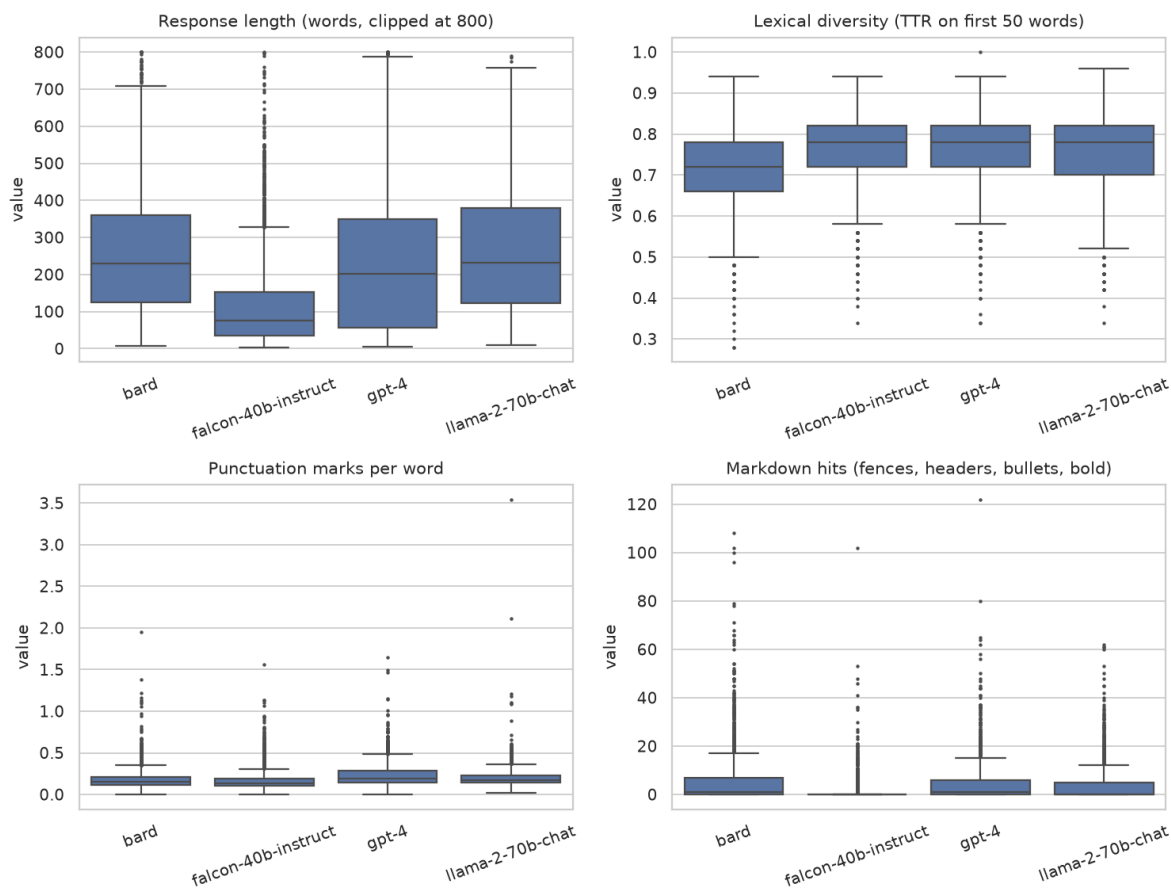


Figure 6: Length separates Falcon. Type-token ratio on 50 words does not. Many responses from every model have no markdown.

Falcon’s median length is 76 words. The others are 230 (Bard), 202 (GPT-4), and 231 (Llama). That is the biggest structural gap. Type-token ratio on a fixed window does not separate them. The means sit between 0.715 and 0.770.

A few phrases show up at very different rates. I counted them on the full balanced set:

- “I’d be happy” appears in 15.6% of Llama responses and in under 1% of Bard and Falcon responses.
- “here’s” appears in 25.0% of Llama responses and 17.8% of GPT-4 responses, but only 1.6% of Bard responses.
- “sure,” appears in 23.5% of Llama responses and 16.2% of Bard responses, and in 3.6% of Falcon responses.
- “as an AI” appears in 8.8% of Falcon responses and rarely for the others.
- A pattern like “80% confidence” appears in about 25% of GPT-4 and Llama responses, 7.3% of Bard, and 4.0% of Falcon. UltraFeedback sometimes asks the model to state a confidence. GPT-4 and Llama do it. Falcon and Bard mostly do not. That is the Chapter 7 point: the same extra instruction does not land the same way on every model.

7.2 Logistic regression weights

Chapter 1 says the weight is the importance of the feature. On the output-only logistic regression, the largest positive weights for each class are:

- Bard: here, main, following, sure, variety, able.
- Falcon: assistant (weight 13.9, the largest single weight I saw), primes, response, correct, sorry, therefore.
- GPT-4: 95, various, remember, now, confidence, another.
- Llama: happy, I'd, here's, 80, confidence, therefore, let's.

Those line up with the phrase counts. Llama is “I’d be happy” and “here’s” and “let’s.” Bard is “sure.” Falcon is “assistant” and “sorry.” The tokens 95, 80, and confidence match the confidence-score habit. I do not think “primes” is a deep style fact. It is probably one repeated answer. Weights can lock onto a rare word. That is a limit of reading importance off a single fit.

7.3 K-means, from Chapter 2

I also clustered the test-set word counts with K-means, $K = 4$, without using the labels. Normalized mutual information against the true model is 0.054. Purity is 0.355. A useless clustering of a balanced four-class problem has purity 0.25, so this is only a little better than chance. The responses do not fall into four blobs in word-count space. The supervised logistic regression still works, because it can lean on a handful of high-weight words. Unsupervised distance does not find the same split. I take that as a warning against saying the four models live in four obvious regions of bag-of-words space.

7.4 Deleting signals and retraining

I retrained the CNN and the LSTM after changing the response.

- **Strip markdown.** Fences, inline code, headers, emphasis, and bullets are removed. The words stay.
- **Fixed length.** Every response becomes exactly 60 words, by cutting or by repeating. Length can no longer separate the classes.
- **Strip punctuation.** Punctuation is deleted. The tokenizer already lowercases.
- **All three.** Markdown, then punctuation, then the 60-word rule.

Table 8: RQ4 average F after each change. The drop is from that model’s original output-only score.

Condition	CNN	CNN drop	LSTM	LSTM drop
Original	0.745	—	0.588	—
Strip markdown	0.741	0.004	0.596	−0.008
Fixed length (60 words)	0.715	0.030	0.602	−0.014
Strip punctuation	0.724	0.021	0.518	0.070
All three	0.680	0.065	0.555	0.033

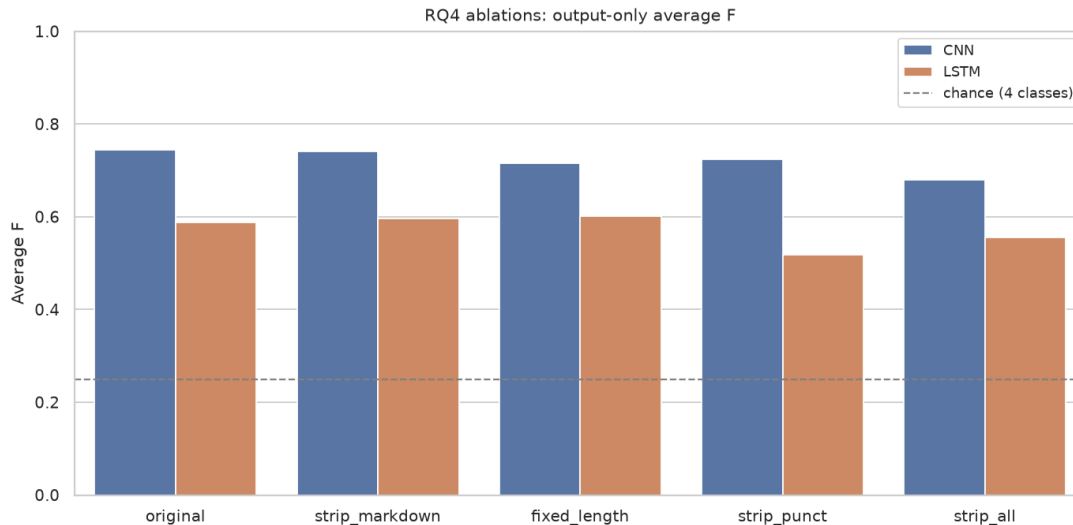


Figure 7: Output-only average F after each ablation.

Markdown markers are not the CNN’s main cue. Removing them costs 0.004. Forcing a common length costs the CNN 0.030. Removing punctuation costs 0.021. All three together cost 0.065, and average F is still 0.680. Word choice is doing real work. That fits the logistic regression result. A linear model on counts, which barely sees order, already scores 0.732.

The LSTM does not use length the way I expected. The 60-word version is slightly better (0.602 versus 0.588). I think a fixed length makes h_T easier, because every example ends after the same number of words. Punctuation matters more for the LSTM. Stripping it drops average F from 0.588 to 0.518. The gates can treat a punctuation mark as a step. When those steps disappear, the final state is less informative.

One training problem: on the markdown ablation, the CNN’s training loss jumped into the hundreds of thousands. Chapter 9 warns that a bad step size makes gradient descent unstable. I did not clip the gradient, because we did not cover that. I still kept the epoch with the best validation accuracy. That epoch’s test average F is the 0.741 in the table. I would not trust the loss curve from that run. I do trust the selected checkpoint more than a diverged one, and the score is close to the original CNN.

8 Lessons

The result I trust most is the pair of checks. Response-only CNN accuracy is 0.745. Instruction-only accuracy is about 0.25 after I closed the Bard/FLAN hole. If those two numbers had been close, I would have been measuring the sampling scheme. Chapter 1’s train/test split does not save you from that. The label can leak through x .

The second lesson is the CNN versus the LSTM, and it matches the Chapter 4 comparison more cleanly than I expected. The CNN sees both sides and only local context, and it wins. The one-direction LSTM sees a long context in theory and, on this data, mostly fails to use it. Reading h_T only, with hidden size 128, was not enough. The bidirectional extra, which the lecture does not teach, got back to 0.717. So the “one side” limitation on that slide is not just a slogan. It showed up in the score.

The third lesson is Chapter 6 and Chapter 7. Holding out writing costs the CNN 0.142 in average F . Holding out code costs 0.076. The model is learning something that transfers, and

something that does not. Llama’s writing and, for the LSTM, Bard’s code are the fragile cases. A detector for a new kind of task needs the held-out number.

I also learned that a linear model from Chapter 1 can beat the LSTM and almost match the CNN. The weights are readable. “Happy,” “here’s,” “assistant,” and “sure” are the kind of habit Chapter 8 is about. K-means on the same counts does not recover the four models (NMI 0.054). Supervision is doing the work. The four models are not an obvious cluster.

Things I would not hide. The task labels are regular expressions. The principle prompts in UltraFeedback blur model style with a dataset choice, and the confidence-score gap is an example. The 60-word ablation repeats short answers, which is a crude way to delete length. The markdown CNN loss exploded. The LSTM test accuracy was lower than its validation accuracy. Domain test sets have a few hundred rows.

If I had more time I would try a smaller learning rate for the CNN, and I would look up gradient clipping, because that one loss curve was not acceptable. I would not switch the main LSTM to bidirectional just to get a higher score. The lecture model is the one-direction one, and the gap is part of the result.

9 LLM tools

An LLM coding assistant helped me write the code, debug training, and draft this report. The accuracies, F scores, count tables, weights, and phrase rates were produced by running that code. They were copied from `results/metrics.json` or counted on `data/processed/samples.csv.gz`. I am responsible for understanding the dataset, the models, the split, and the claims in this write-up before I submit.

10 How to rerun

From the repository root, with the packages in `requirements.txt`:

```
python -m src.prepare_data
python -m src.run_all
cd report && pdflatex report.tex
```

`prepare_data` downloads UltraFeedback only if the raw files are missing. The processed table used here is already in the repo. `run_all` retrains the tables above at seed 42 on CPU. The committed `results/metrics.json` is the run this report describes.

References

- [1] Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Wei Zhu, Yuan Ni, Guotong Xie, Zhiyuan Liu, and Maosong Sun. UltraFeedback: Boosting language models with high-quality feedback. arXiv:2310.01377, 2023.